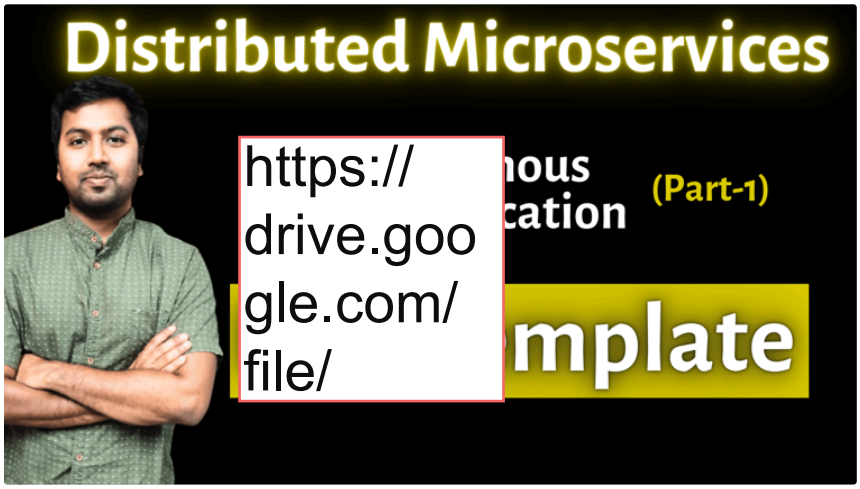
RestClient - Synchronous Communication

Prerequisite to understand RestClient is, to first understand its predecessor RestTemplate



Kindly check the Part-1 (RestTemplate), if there is any doubt with that topic.

In previous video, we saw the Limitation of RestTemplate:

- In RestTemplate, there are already so many overloaded methods, so its hard to remember and maintain.
- RestTemplate was build before concepts like Retry, circuit breaker etc.. So adding support means more overloaded methods and not user friendly.
- RestTemplate is in Maintenance mode - means no new feature, only bug fixes.

Alternative of RestTemplate: **RestClient**

- Introduced in Spring Framework 6.0+ and SpringBoot 3.0+
- Synchronous/blocking in nature, means client wait for the response from the Server before continuing.
- Modern, fluent based API that is more readable and easy to maintain.

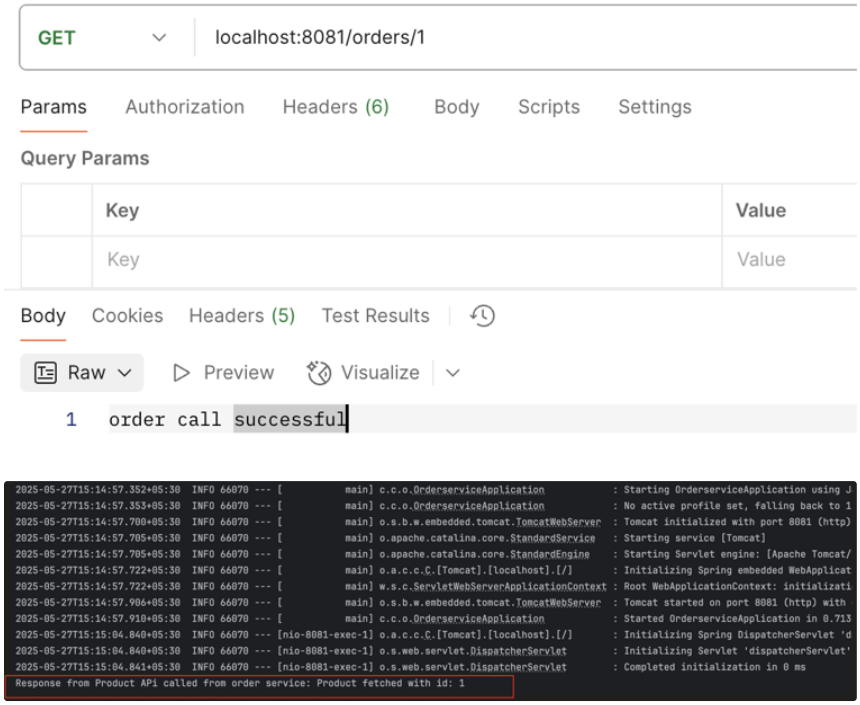
OrderService needs to invoke ProductService

OrderService (port 8081) ----------⟶ **ProductService** (port 8082)

```
@Configuration
public class AppConfig {

    @Bean
    public RestClient restClientInstance() {
        return RestClient.create();
    }
}

```

```java
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable String id) {
        return "Product fetched with id: " + id;
    }
}

```

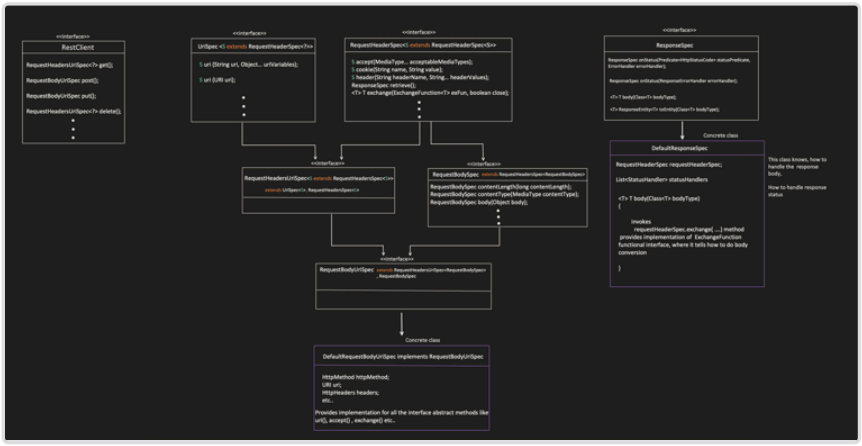
```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestClient restClient;

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        String response = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .retrieve()
                .body(String.class);

        System.out.println(
                "Response from Product API called from order service: " + response
        );

        return ResponseEntity.ok("order call successful");
    }
}

```

So, first important thing to understand is, how this Fluent API works?

- Fluent API means chaining of method calls.

- Each method call returns an object (of next step) that exposes next set of operations (methods).

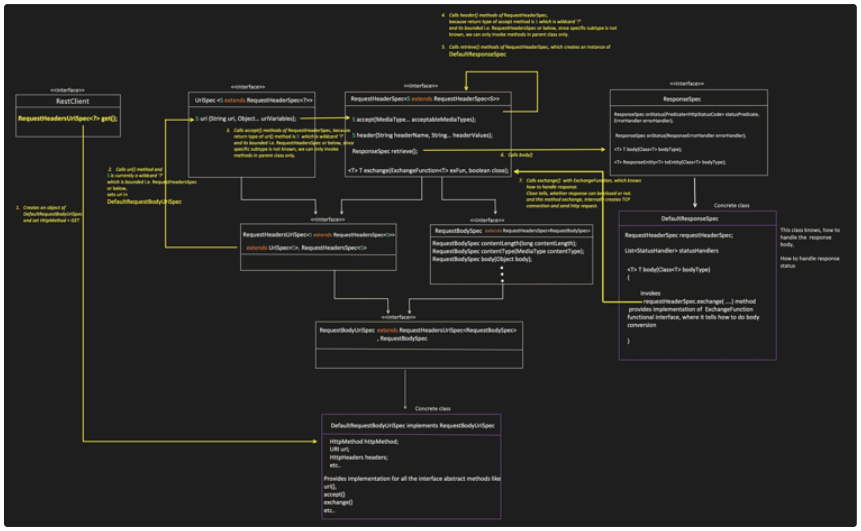


I am assuming that, we all know about Generics Class and Wildcards in Java

Now, lets see how Fluent API works for GET, POST, DELETE operation.

GET:

```
RestClient restClient = RestClient.create();

String response = restClient
        .get()
        .uri("http://localhost:8082/products/" + id)
        .accept(MediaType.APPLICATION_JSON)
        .header("X-Custom-Header", "xyz")
        .retrieve()
        .body(String.class);


```

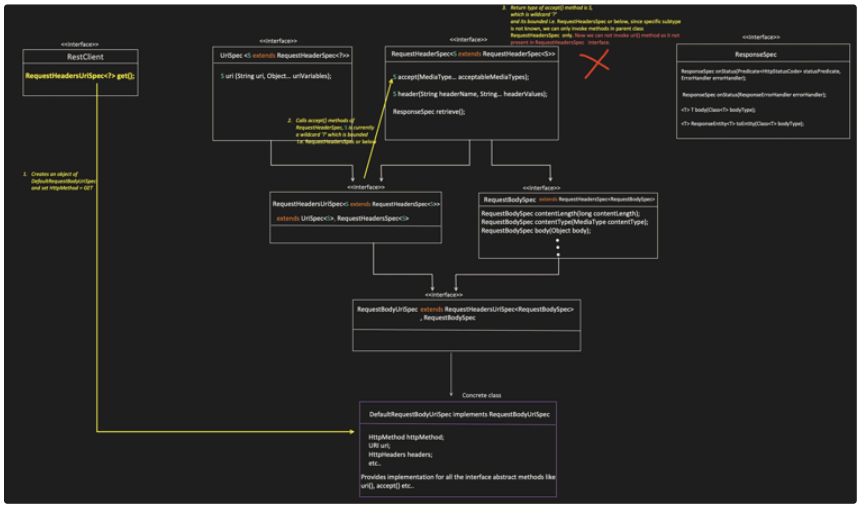


Let see, how flow can be broken

GET

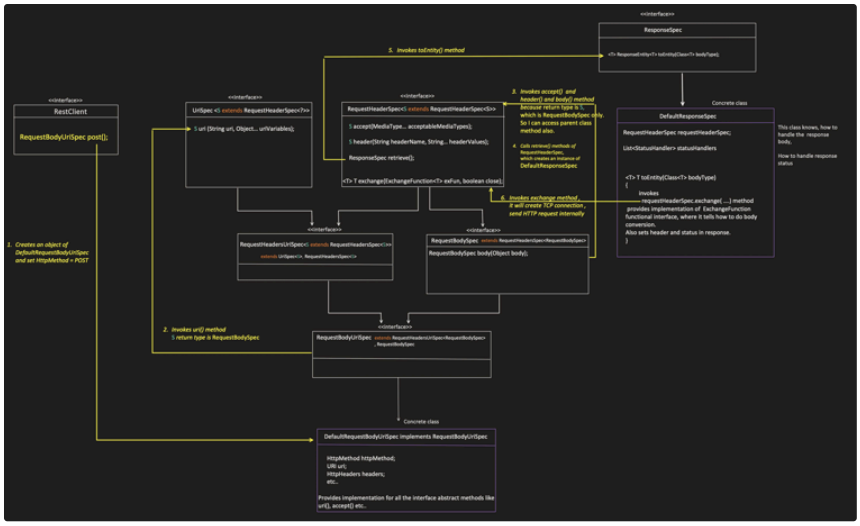
```
RestClient restClient = RestClient.create();

String response = restClient
        .get()
        .accept(MediaType.APPLICATION_JSON)
                .uri
```

Instead of uri method first, I invoked the accept method first and now I can not set the URI itself. Why, does sequence of call matters?



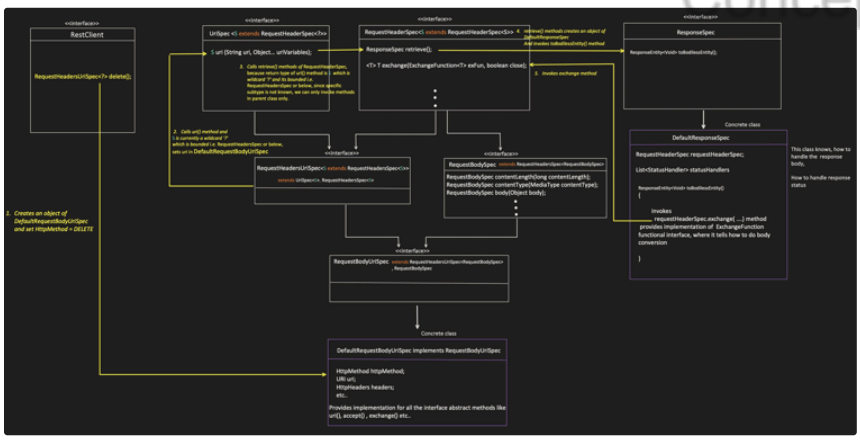
POST:

```java
RestClient restClient = RestClient.create();

ResponseEntity<ProductEntity> response = restClient
        .post()
        .uri("http://localhost:8082/products/create")
        .accept(MediaType.APPLICATION_JSON)
        .header("Content-Type", "application/json")
        .body(new ProductEntity("Ice-cream")) // some new object which need to be created
        .retrieve()
        .toEntity(ProductEntity.class);

ProductEntity responseBody = response.getBody();

```



DELETE:

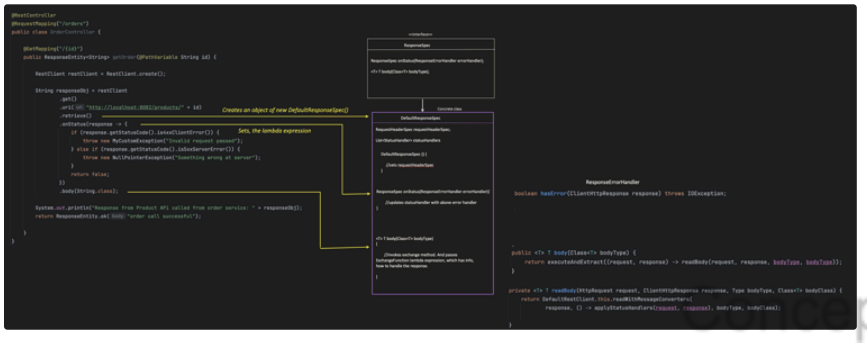
```
RestClient restClient = RestClient.create();

ResponseEntity<Void> response = restClient
        .delete()
        .uri("http://localhost:8082/products/" + id)
        .retrieve()
        .toBodilessEntity();

HttpStatusCode deletionStatus = response.getStatusCode();

```

Exception Handling:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        RestClient restClient = RestClient.create();

        String responseObj = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .retrieve()
                .onStatus(response -> {
```

```
15
16                      if (response.getStatusCode().is4xxClientError()) {
17                          throw new MyCustomException("Invalid request
    passed");
18                      } else if
    (response.getStatusCode().is5xxServerError()) {
19                          throw new NullPointerException("Something
    wrong at server");
20                      }
21
22                      return false;
23                  })
24                  .body(String.class);
25
26          System.out.println(
27                  "Response from Product API called from order service:
    " + responseObj
28          );
29
30          return ResponseEntity.ok("order call successful");
31      }
32  }
33
```



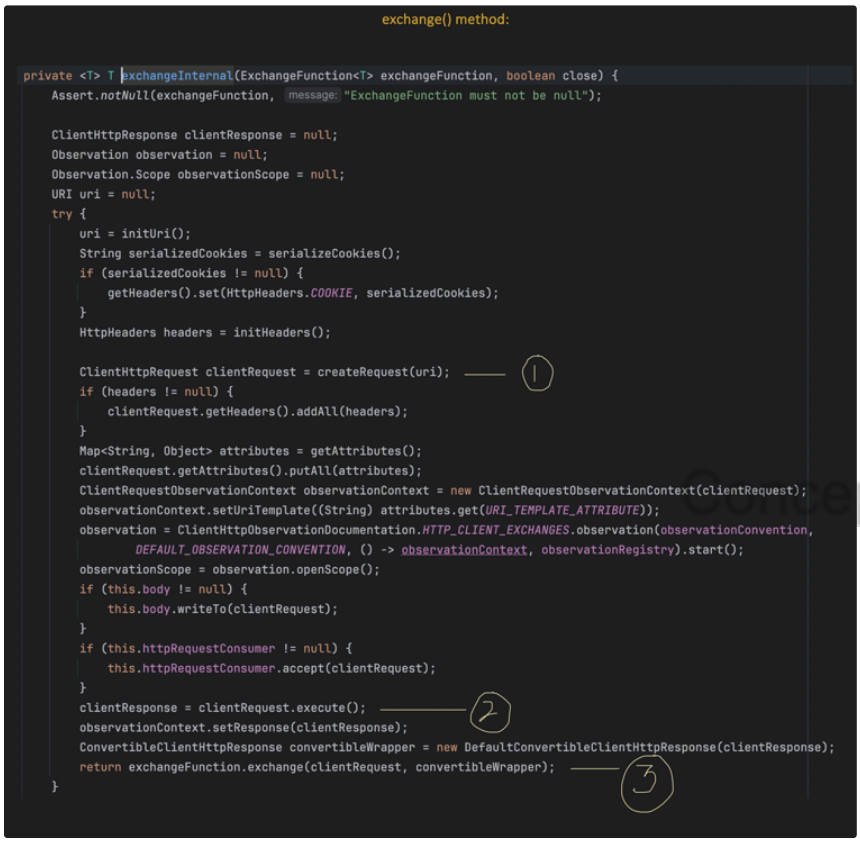
If we want full control over Response building and Exception handling

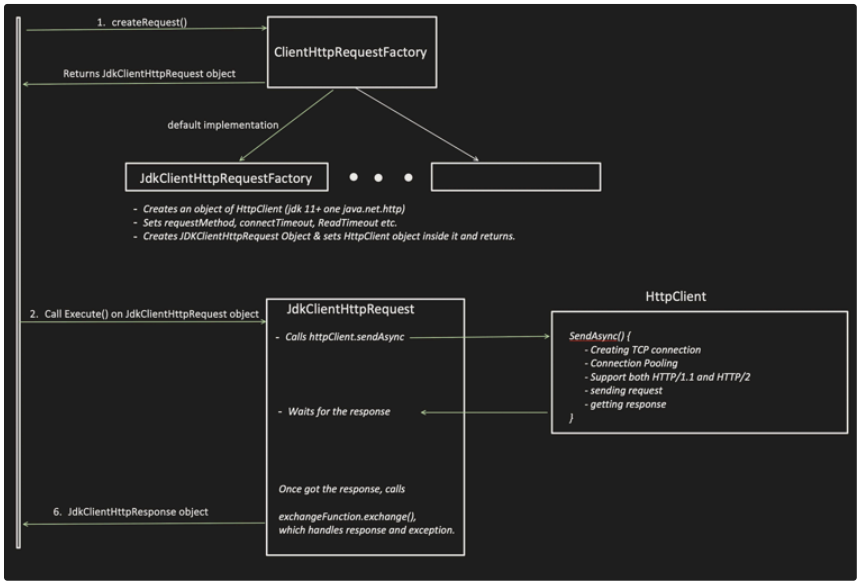
Then we can directly call exchange method, instead of relying of ResponseSpec to do that.

```
1  @RestController
2  @RequestMapping("/orders")
3  public class OrderController {
4
5      @GetMapping("/{id}")
6      public ResponseEntity<String> getOrder(@PathVariable String id) {
7
8          RestClient restClient = RestClient.create();
9
10         String responseObj = restClient
11                 .get()
12                 .uri("http://localhost:8082/products/" + id)
13                 .exchange((request, response) -> {
14
15                     if (response.getStatusCode().is4xxClientError()) {
16                         throw new MyCustomException("Invalid request
    passed");
17                     } else if
    (response.getStatusCode().is5xxServerError()) {
18                         throw new InternalError("Something wrong at
    server");
19                     } else {
20                         // response mapping logic if no error
21                         return StreamUtils.copyToString(
22                                 response.getBody(),
23                                 StandardCharsets.UTF_8
24                         );
```

```
                    }
                });

        System.out.println(
                "Response from Product API called from order service: " + responseObj
        );

        return ResponseEntity.ok("order call successful");
    }
}

```

exchange() method:





Adding Interceptors

OrderService:

```
public class MyCustomRequestInterceptor implements
ClientHttpRequestInterceptor {

    @Override
    public ClientHttpResponse intercept(
            HttpRequest request,
            byte[] body,
            ClientHttpRequestExecution execution
    ) throws IOException {

        request.getHeaders().add("X-Custom-Header", "myvalue");
        return execution.execute(request, body);
    }
}


```

```
@Configuration
public class AppConfig {

    @Bean
    public RestClient restClientInstance(
            ClientHttpRequestInterceptor myCustomInterceptor
    ) {
        return RestClient.builder()
                .requestInterceptor(myCustomInterceptor)
                .build();
    }

    @Bean
    public ClientHttpRequestInterceptor customRequestInterceptor() {
        return new MyCustomRequestInterceptor();
    }
}

```



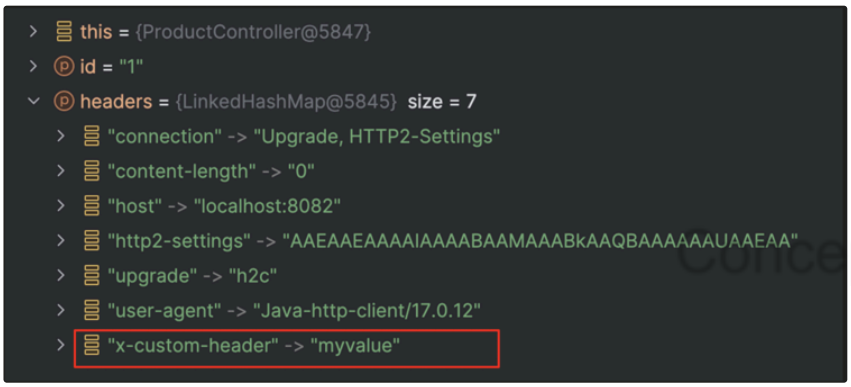
```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestClient restClient;

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        ResponseEntity<String> responseEntityObj = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .retrieve()
                .toEntity(String.class);

        System.out.println(
                "Response from Product API called from order service:
"
                        + responseEntityObj.getBody()
        );

```

```
        return ResponseEntity.ok("order call successful");
    }
}

```

ProductService

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable String id) {
        return "Product fetched with id: " + id;
    }
}

```

When product service got invoked, able to see the header which we added using interceptor



Interceptor invokes at step2